Laborator 7 - Documentație. Patterns. Modele de dezvoltare. Evaluarea rezultatelor proiectului.

Documentația

Orice aplicație software dispune de mai multe forme de documentație. Principalele forme sunt:

- documentația utilizatorului (*user documentation* sau *user manual*)
- documentația tehnică a produsului.

Pe parcursul dezvoltării aplicației se pot genera și alte forme de documentație: raportare, administrativă, de natură legală.

Persoanele care se ocupă de realizarea documentației tehnice a unei aplicații / a unui produs sunt numite *technical writers*. Documentația tehnică va cuprinde cerințele software, proiectarea sistemului și arhitectura sistemului. Documentația tehnică este utilă utilizatorului avansat sau administratorului sau unui membru al serviciului de mentenanță a aplicației. Scopul acestei documentații este de a oferi informații/detalii despre funcționarea internă a aplicației și despre legătura între componente.

Documentația de utilizator este, în general, descrisă în cadrul unui manual de utilizare a aplicației. Documentația este destinată oricărui tip de utilizator și este orientată pe a prezenta funcționalitățile aplicației și modul de folosire a acesteia fără a insista pe modul în care aceasta este proiectată. Exceptând formatele standard de manual de utilizare, formele online de ajutor (forumuri, liste de discutii), FAQ-uri, how-to-uri, tutoriale, pot fi, de asemenea considerate documentație de utilizator.

În general, documentația se realizează pe un set de șabloane (template-uri) predefinite. Totuși, o dată cu dezvoltarea Web-ului, un număr din ce în ce mai mare de articole de documentație se găsesc pe wiki-uri, pagini web specializate, blog-uri, în special cele destinate dezvoltatorilor. Forme de documentare precum MSDN Library [http://msdn.microsoft.com/en-us/library/default.aspx], Java Documentation [http://java.sun.com/javase/6/docs/api/], GNU libc [http://www.gnu.org/software/libc/manual/] sunt accesibile online.

Generarea documentației tehnice

În proiectele software, un rol foarte important pentru reușita aplicației și buna înțelegere între membrii echipei este modul în care se documentează funcțiile, metodele, clasele și bibliotecile expuse. Pentru aplicații de tip bibliotecă software, este necesar ca utilizatorului acestora (un dezvoltator) să îi fie bine definite și documentate funcțiile și interfețele expuse.

Forme de documentare a codului, interfețelor includ pagini de manual, pagini web, tutoriale, documente LaTeX, references etc. În general, majoritatea formelor de documentare utilizate oferă o interfață web (MSDN library [http://msdn.microsoft.com/en-us/default.aspx], pagini de manual [http://linuxmanpages.com/], pagini info [http://www.gnu.org/manual/], reference libc [http://www.gnu.org/software/libc/manual/html_node/index.html], Java API [http://java.sun.com/javase/6/docs/api/], Python Standard Library [http://docs.python.org/library/], PHP [http://www.php.net/manual/en/index.php]) pentru a facilita accesul și citirea acestora.

O formă posibilă de documentare a codului, utilizată cu succes de Java API [http://java.sun.com/javase/6/docs/api/] este generarea automată de documentație din surse folosind utilitarul Javadoc [http://java.sun.com/j2se/javadoc/]. Folosind un stil special de comentare a surselor, care nu afectează lizibilitatea codului, aplicațiile de generare automată a documentației obțin, în diverse formate, cel mai răspândit fiind HTML, informații despre utilizarea funcțiilor/claselor/metodelor expuse.

Javadoc

Javadoc [http://docs.oracle.com/javase/7/docs/technotes/guides/javadoc/index.html] este o aplicație produsă de Sun Microsystems care permite generarea de documentație automată pentru surse Java. O descriere amplă a tag-urilor care sunt folosite pentru obținerea documentației găsiți aici [http://java.sun.com/j2se/1.5.0/docs/tooldocs/windows/javadoc.html]. În exemplul de mai jos este prezentat modul în care este folosit Javadoc pentru generarea documentației pentru acest fișier Java [http://elf.cs.pub.ro/mps/res/laboratoare/laborator-06/mpstest.java]. Rezultatul obținut este prezentat aici [http://elf.cs.pub.ro/mps/labs/lab-06/javadoc/].

```
razvan@valhalla:~school/2009-2010/mps/labs/lab-06$ javadoc -author -version MpsTest.java
Loading source file MpsTest.java...
Constructing Javadoc information...
MpsTest.java:16: cannot find symbol
symbol  : class myException
location: class MpsTest
        public MpsTest() throws myException
                                ^
Standard Doclet version 1.6.0_16
[...]
```

Doxygen

Doxygen [http://www.stack.nl/~dimitri/doxygen/] permite generarea automată a documentației într-o diversitate de formate pentru un număr mare de limbaje de programare.

Aici [http://elf.cs.pub.ro/mps/labs/lab-06/doxygen-python/] găsiți documentația generată pentru acest fișier Python [http://elf.cs.pub.ro/mps/res/laboratoare/laborator-06/mpstest.py] folosind acest fișier de configurare [http://elf.cs.pub.ro/mps/res/laboratoare/laborator-06/mps-doxy.conf].

Pentru obținerea documentației se urmează trei pași:

1. generarea unui fișier de configurare:

```
razvan@valhalla:~school/2009-2010/mps/labs/lab-06$ doxygen -g mps-doxy.conf

Configuration file `mps-doxy.conf' created.

Now edit the configuration file and enter

  doxygen mps-doxy.conf

to generate the documentation for your project
```

2. editarea fișierului de configurare; directiva INPUT este folosită pentru a selecta fișierele/directoarele care se doresc parsate
3. rularea doxygen pentru obținerea documentației:

```
razvan@valhalla:~school/2009-2010/mps/labs/lab-06$ doxygen mps-doxy.conf
Notice: Output directory `out-doxy/' does not exist. I have created it for you.
Searching for include files...
Searching for example files...
Searching for images...
Searching for dot files...
[...]
```

Patterns și anti-patterns

Domeniul ingineriei software are o dinamică deosebită. Fiecare limbaj de programare sau paradigmă înseamnă folosirea altui set de reguli și noțiuni. Cu toate acestea există un set de reguli de bază care trebuie urmărite pentru a asigura succesul unei aplicații. Aceste reguli au ca scop integrarea facilă a aplicației cu alte aplicații, folosirea de interfețe ușor de înțeles, adăugarea ușoară de noi caracteristici. Un set de caracteristici pe care trebuie să le îndeplinească o aplicație sau un modul sunt:

- **utilitatea** - aplicația trebuie sa fie utilă, să aibă sens crearea și folosirea acesteia;
- **simplitatea** - evitarea complexității inutile (feature creep [http://en.wikipedia.org/wiki/Feature_creep] sau software bloat [http://en.wikipedia.org/wiki/Software_bloat]);
- **claritatea** - interfața modulului/aplicației trebuie sa fie ușor de înțeles;
- **generalitatea** - modulul trebuie să aibă un comportament așteptat într-un număr cât mai mare de situații și trebuie să se adapteze;
- **automatizarea** - modulul respectiv trebuie să execute o acțiune care să elibereze o persoană de un efort ulterior (folosită în scripting).

Caracteristicile de mai sus sunt descrise în cartea The Practice of Programming [http://books.google.com/books?id~to6M9_dbjosC&lpg~PP1&dq~Practice%20of%20programming&pg~PP1#v~onepage&q~lulu&f~false] de Brian Kernighan și Rob Pike.

Aceste caracteristici sunt generice la nivelul oricărui limbaj de programare. Apariția limbajelor de nivel foarte înalt [http://en.wikipedia.org/wiki/Very_high-level_programming_language] oferă un context prin care dezvoltatorul nu trebuie să se preocupe de modul de proiectare/dezvoltare a aplicației. Aceste limbaje sunt, însă, foarte specifice astfel încât pentru majoritatea limbajelor este nevoie ca dezvoltatorul să aibă cunoștințe și experiențe în folosirea tehnicilor de tip “best practice” în inginerie software.

Patterns & best practices

Tehnicile de tip “best practices” și ”(design) patterns” se referă la mecanisme de proiectare și implementare a unei aplicații/modul pentru ca aceasta să respecte cât mai bine caracteristicile de claritate, generalitate, simplitate. În acest sens se definesc o serie de principii/mecanisme pe care dezvoltatorul/programatorul este recomandat să le urmeze.

Noțiunea de “design pattern” (șablon de proiectare) se referă în special la limbajele orientate obiect. Un design pattern este o formă de abstractizare a unei soluții la o problemă într-o formă generică pentru a permite utilizarea soluției în alte situații. Exemple de design patterns [http://en.wikipedia.org/wiki/Design_pattern_(computer_science)] sunt:

- Abstract factory [http://en.wikipedia.org/wiki/Abstract_factory_pattern] - folosirea unei clase abstracte/interfețe care grupează alte fabrici (factories) de obiecte;
- Lazy Initialization [http://en.wikipedia.org/wiki/Lazy_initialization] (tehnică folosită și în alte situații) - întârzierea creării unui obiect până în momentul în care este folosit;
- Singleton [http://en.wikipedia.org/wiki/Singleton_pattern] - permite instanțierea unui singur obiect dintr-o clasă dată;
- Iterator [http://en.wikipedia.org/wiki/Iterator_pattern] - folosirea unui mecanism de parcurgere secvențială a unui obiect agregat fără a expune reprezentarea internă;
- Null Object [http://en.wikipedia.org/wiki/Null_Object_pattern] - obiect cu valoarea neutră (nulă);
- Visitor [http://en.wikipedia.org/wiki/Visitor_pattern] - separarea implementării unui algoritm de obiectul asupra căruia acționează;
- Decorator [http://en.wikipedia.org/wiki/Decorator_pattern] - adăugarea unor noi funcționalități unui obiect existent în mod dinamic.

La un nivel generic, există noțiune de “development methodology/philosophy” sau tehnici/patterns pentru dezvoltarea aplicațiilor cu un nivel de genericitate ridicat - folosite de un spectru larg de limbaje de programare, nu doar cele orientate obiect. Între acestea amintim:

- Code and fix [http://en.wikipedia.org/wiki/Code_and_fix] este o tehnică de eliminare a fazei de proiectare pentru module mici; această metodă presupune implementarea și testarea imediată modulului;
- Continuous integration [http://en.wikipedia.org/wiki/Continuous_integration] se referă la folosirea de tehnici care să minimizeze timpul de integrare; tehnica presupune colaborarea membrilor unei echipe prin folosirea unui repository, a unor aplicații de compilare automată și prin realizarea de commit-uri dese și funcționale în repository;
- Don't repeat yourself [http://en.wikipedia.org/wiki/Don%27t_repeat_yourself] (DRY) se referă la implementarea unei funcționalități într-un singur modul de unde va fi folosită de celelalte module; în acest fel, modificările se vor realiza într-un singur loc fără a necesita schimbări în alte zone;
- Hollywood principle [http://en.wikipedia.org/wiki/Hollywood_principle] (*don't call us, we'll call you*)/Inversion of control [http://en.wikipedia.org/wiki/Inversion_of_control] (IOC) se referă la inversarea ordinii uzuale de apel între module; modulul inferior în ierarhie poate apela modulul superior pentru a facilita efortul de implementare; folosirea de callback-uri sau evenimente reprezintă forme ale acestui principiu;
- KISS (Keep it simple, stupid) [http://en.wikipedia.org/wiki/KISS_principle] accentuează importanța simplității, care ar trebui să fie un țel important al aplicației și că aspectele de complexitate nenecesară trebuie evitate;
- MoSCoW [http://en.wikipedia.org/wiki/MoSCoW_Method] este o metodă de prioritizare a acțiunilor în cadrul unui proces de dezvoltare; majusculele din MoSCoW se referă la cele 4 clase de priorități: **M** - Must have this, **S** - Should have this, **C** - Could have this if doesn't affect anything else, **W** - Won't have this time, but Would like in the future;
- Quick-and-dirty [http://en.wikipedia.org/wiki/Quick_and_dirty], de obicei conotată negativă, se referă la implementarea rapidă a unei metode/clase/modul fără a ține cont de nivelul de generalitate a acesteia; este utilă în situații în care se dorește testarea/verificarea unei anumite componente și nu este justificată implementarea unei metode cât mai bune;
- Release early, release often [http://en.wikipedia.org/wiki/Release_early_release_often] (RERO) încurajează livrările frecvente și timpurii ale unei aplicații pentru a crea o buclă de feedback între utilizatori/testeri și dezvoltatori; se permite astfel dezvoltarea rapidă a aplicației în conformitate cu cerințele utilizatorilor pentru software;
- You ain't gonna need it [http://en.wikipedia.org/wiki/You_ain%27t_gonna_need_it] (YAGNI) este o tehnică folosită în extreme programming [http://en.wikipedia.org/wiki/Extreme_programming] conform căreia programatorii nu ar trebui să adauge funcționalitate până când nu este necesară;
- Separation of concerns [http://en.wikipedia.org/wiki/Separation_of_concerns] (SoC) se referă la împărțirea unei aplicații în componente cu funcționalități distincte, care nu se suprapun; tehnici de bază pentru obținerea SoC sunt modularitatea și încapsularea.

Un set important de principii, menționate în The Practice of Programming [http://books.google.com/books?id~to6M9_dbjosC&lpg~PP1&dq~Practice%20of%20programming&pg~PP1#v~onepage&q~lulu&f~false] se referă la proiectarea interfeței expuse de un modul. O interfață proiectată corespunzător va facilita integrarea altor module și scalarea facilă a aplicației.

- ascunderea detaliilor de implementare (încapsulare, abstractizare)
- alegerea unui set ortogonal de primitive
- nu lucra pe la spatele utilizatorului
- același lucru trebuie realizat în toate situațiile (consecvență)

Anti-patterns

În ingineria programării anti-patterns sunt metode de proiectare (organizațional, software etc.) care, deși par evidente și simple sunt de fapt greșite, mai complexe decât e necesar sau suboptime. O listă de anti-pattern-uri clasice se găsesc pe wikipedia [http://en.wikipedia.org/wiki/Anti-pattern]. Câteva anti-pattern-uri frecvente printre programatorii mai puțin experimentați sunt:

- Call Super [http://en.wikipedia.org/wiki/Call_super] - necesitatea de a chema o metodă suprascrisă dintr-o clasa de bază;
- God Object [http://en.wikipedia.org/wiki/God_object] - existența unui obiect cu o functionalitate foarte largă;
- Object Orgy [http://en.wikipedia.org/wiki/Object_orgy] - delimitare neconcludentă a variabilelor interne a unui obiect;
  - se recomandă ca variabilele interne să fie accesate prin mutatori și accesori (*white listing*);
  - de exemplu, în Java:

```
void setVisible(boolean);    // mutator
boolean isVisible();         // accesor
```

- Sequential Coupling [http://en.wikipedia.org/wiki/Sequential_coupling] - necesitatea de a chema anumite metode într-o anumita ordine. Metodele respective pot fi incluse în altă metodă care să le apeleze in ordinea corectăl;
- Yet Another Useless Layer - adăugare de niveluri de abstractizare nenecesare;
- Blind faith [http://en.wikipedia.org/wiki/Blind_faith_(computer_science)] - încrederea că o anumită funcție este corectă fără a fi testată;
- Accumulate and fire - transmiterea de argumente unei funcții folosind variabile globale, nu parametri;
- Voodoo programming [http://en.wikipedia.org/wiki/Voodoo_programming] & Deep magic [http://en.wikipedia.org/wiki/Black_magic_(programming)] - implementarea unei soluții vag înțelese sau folosirea de utilitare necunoscute (vezi http://www.codemaestro.com/reviews/9 sqrt [http://www.codemaestro.com/reviews/9 sqrt]);
- Magic numbers [http://en.wikipedia.org/wiki/Magic_number_(programming)#Unnamed_numerical_constant] & strings - folosirea constantelor și a sirurilor de caractere fără o explicație (un comentariu sau denumire de variabilă);
- Copy and paste programming [http://en.wikipedia.org/wiki/Copy_and_paste_programming] - lipsa unei generalizări și copierea codului cu mici modificări (vezi Blind Faith);
- Reinventing the wheel [http://en.wikipedia.org/wiki/Reinventing_the_wheel] - nefolosirea codului deja existent (cum ar fi third-party software);
- Programming by permutation [http://en.wikipedia.org/wiki/Programming_by_permutation] - încercarea de a repara un bug prin mici modificări aleatoare fără a înțelege exact unde este greșeala.

Modele de dezvoltare

Modelele/Metodologiile de dezvoltare sunt, în general, clasificate după secvențialitatea acțiunilor. Există, astfel, două clase mari de metodologii de dezvoltare: liniare și iterative. În cadrul celor liniare, fiecare etapă a proiectului se bazează pe etapa anterioară. În cazul modelelor iterative, se realizează o primă parte a etapelor de proiectare, implementare și testare urmând ca, ulterior, să se treacă la faza următoare.

Modelul Waterfall este reprezentantul clasic al metodologiilor liniare. Fazele unui proiect (cerințe, proiectare, implementare, verificare, validare) se succed. Deși oferă o organizare a etapelor ce compun activitățile unui proiect, modelul Waterfall este în general privit ca o opțiune nepotrivită în cadrul proiectelor software. Principalul său dezavantaj este rigiditatea. Majoritatea proiectelor software nu pot defini de la bun început contextul în care se va implementa aplicația. Schimbarea rapidă a condițiilor, constrângerilor, opțiunilor pe parcursul desfășurării activităților proiectului impune iterarea procesului de proiectare/implementare/testare. O extensie a modelului Waterfall este modelul V [http://en.wikipedia.org/wiki/V-Model_(software_development)].

Prototiparea software presupune crearea de prototipuri, versiuni incomplete ale programului care va fi realizat. Aceste prototipuri vor fi analizate și evaluate. Unele prototipuri vor fi aruncate, altele vor fi modificate, iar altele pot asigura baza pentru implementarea efectivă a unui modul/componente. RAD (Rapid Application Development) este o metodologie de dezvoltare software care include dezvoltarea iterativă și crearea de prototipuri.

Modelul spirală presupune realizarea unor cicluri de proiectare și prototipizare. În fiecare ciclu se parcurg 4 faze: determinarea obiectivelor, evaluarea riscurilor, dezvoltarea și verificarea livrabilelor, planificarea pentru următoarea iterație. Modelul spirală este, în general, folosit de companiile mari și activitățile complexe.

Modelul de dezvoltare agilă (agile software development) se referă la un set de tehnici bazate pe dezvoltare iterativă, în care soluțiile și cerințele evoluează prin intermediul colaborării între membrii diverselor echipe. Metodele agile încurajează inspecțiile frecvente, munca în echipă, auto-organizarea, responsabilitatea. Agile Manifesto a menționat noile valori ale modelului:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation

- Responding to change over following a plan

Scrum [http://en.wikipedia.org/wiki/Scrum_(development)] este o metodă iterativă/incrementală pentru gestiunea unei aplicații complexe, folosită, în general, la un loc cu agile development. Scrum presupune o serie de întâlniri/sprint-uri colaborative precum: daily scrum, scrum of scrums, sprint planning meeting, sprint-ul efectiv, sprint review meeting.

Un codesprint sau hackathon [http://en.wikipedia.org/wiki/Hackathon] sau codefest este o activitate în care un grup de dezvoltatori se întâlnesc și programează colaborativ. Un hackathon presupune prezența unui număr considerabil de programatori și desfășurarea de activități pe parcursul câtorva zile până la maxim o săptămână.

Interfețe cu utilizatorul

Cele două forme comune de interfață cu utilizatorul sunt interfața mod text (CLI - *Command Line Interface*) și interfața mod grafic (GUI - *Graphical User Interface*). O dată cu dezvoltarea Internet-ului, se poate considera o nouă formă de interfață de natură grafică, denumită WebUI - *Web User Interface*.

Evoluția și răspândirea sistemelor de calcul și a aplicațiilor au dus la o creștere accelerată a interfețelor grafice și web în ultimele două decenii. Preponderent aplicațiile vor oferi o interfață grafică, deși interfața în linia de comandă rămâne utilizată pentru automatizare și eficiență sau de persoane cu profil tehnic (dezvoltatori, administratori de rețea/sistem).

Nivelul de intuitivitate, ușurință în folosire și eficiență a celor trei tipuri de interfețe variază dar există un set de decizii de proiectare care trebuie avute în vedere în momentul utilizării unei anumite interfețe.

Regula importantă pe care trebuie să o respecte orice interfață, indiferent de tipul acesteia este “oferă-i utilizatorului ceea ce se așteaptă”. O interfață care va inversa rolurile butoanelor *OK* și *Cancel* va încurca utilizatorul.

Command Line Interface

O aplicația care oferă o interfață în linia de comandă, va avea, în funcție de natura aplicației, un număr mai mare sau mai mic de argumente. În general argumentele sunt:

- **argumente “pure”**: nume de fișiere, nume de utilizatori, identificatori
- **opțiuni**; opțiunile pot fi clasificate
  - în funcție de prezența unui argument
    - opțiuni simple (`ls -a`, `ps -e`);
    - opțiuni cu argumente `lsof -p 1001` (opțiunea `-p` este asociată cu un pid ce reprezintă PID-ul unui proces);
  - în functie de “lungimea argumentului”
    - opțiuni format scurt (`-t`, `-l`, `-p`, `-n`);
    - opțiuni format lung (`--tcp`, `--listening`, `--program`, `--numeric`).

Argumentele trebuie să aibă nume intuitive și să permită folosirea tuturor opțiunilor importante ale aplicației. Opțiunile trebuie documentate și se recomandă o formă de ajutor rapid în acest sens (`/?` pe Windows sau `--help` pe Unix). Pentru parsarea argumentelor se recomandă folosirea bibliotecii getopt [http://www.gnu.org/s/libc/manual/html_node/Getopt.html], disponibilă și pentru Python [http://docs.python.org/library/getopt.html], shell [http://opengroup.org/onlinepubs/007908775/xcu/getopts.html], Perl [http://perldoc.perl.org/Getopt/Long.html], PHP [http://php.net/manual/en/function.getopt.php], Ruby [http://ruby-doc.org/stdlib/libdoc/getoptlong/rdoc/index.html].

Graphical User Interface

O interfață grafică (GUI) folosește elemente descrise de WIMP: window, icon, menu, pointing device. Interfața grafică permite folosirea obiectelor anteriore pentru a obține un rezultat sau efect.

Interfața grafică are ca principal scop oferirea unei interfețe ușor de folosit și ergnomice. Paleta de culori folosită, fonturile, dimensiunea obiectelor și plasarea acestora sunt criterii importante pentru definirea interfeței grafice.

Aplicațiile grafice masive (de tipul desktop environment) dispun, în general, de un set de documente denumite human interface guidelines [http://en.wikipedia.org/wiki/Human_interface_guidelines]. Acestea oferă dezvoltatorilor și proiectanților un set de recomandări pentru a îmbunătăți experiența utilizatorilor. Aceste recomandări dictează regulile de utilizabilitate a aplicației. Exemple sunt:

- Apple Human Interface Guidelines [http://developer.apple.com/mac/library/documentation/UserExperience/Conceptual/AppleHIGuidelines/XHIGIntro/XHIGIntro.html]
- Windows Vista Human Experience Guidelines [http://msdn2.microsoft.com/en-us/library/Aa511258.aspx]
- GNOME Human Interface Guidelines [http://library.gnome.org/devel/hig-book/stable/]

Implementarea interfețelor grafice este facilitată de prezența toolkit-urilor grafice [http://en.wikipedia.org/wiki/Widget_toolkit], denumite și *widget toolkit*, precum Microsoft Foundation Class [http://en.wikipedia.org/wiki/Microsoft_Foundation_Class] pe Windows, GTK+ [http://en.wikipedia.org/wiki/GTK%2B] pentru aplicații GNOME, Qt [http://en.wikipedia.org/wiki/Qt_(toolkit)] pentru KDE, wxWidgets [http://en.wikipedia.org/wiki/WxWidgets] ca interfață portabilă între platforme.

Web User Interface

Interfețele web (WebUI, WUI) sunt interfețele folosite în cadrul browserelor de paginile Web din Internet. O interfață web funcționează în contextul unui sistem client-server în care serverul este furnizorul paginii web care trebuie redată în client (browser). Pe lângă folosirea HTML ca mecanism de bază de reprezentare a paginii și comunicare client/server, noile aplicații dispun de tehnologii precum Ajax, Adobe Flex etc.

O interfață web trebuie să respecte, de asemenea, standarde de utilizabilitate. Totuși, o interfață web dispune de constrângeri suplimentare precum faptul că trebuie să funcționeze în contextul mai multor browsere și trebuie să țină cont de conexiunea/comunicarea client/server.

Evaluarea rezultatelor proiectului

Evaluarea rezultatelor proiectului Pentru a evalua un proiect cel mai adesea se folosesc diverse metrici care să permită o analiză a rezultatelor obținute atât din punct de vedere cantitativ cât și calitativ.

Analiza cantitativă a rezultatelor presupune determinarea performanței soluției software dezvoltate în condiții ideale de rulare (ex: pentru o aplicație client-server se presupune că aplicația client comunică fără probleme cu server-ul, nu există delay etc.). Pentru o astfel de analiză ne interesează o acuratețe cât mai mare a rezultatelor oferite de un anumit algoritm, timpi cât mai buni de rulare etc. Este important ca pentru astfel de metrici să fie menționate condițiile în care a fost rulată aplicația (hardware, seturi de date etc.)

Analiza calitativă are mai mult o natură subiectivă și presupune evaluarea soluției propuse pe baza unor intrări și metrici care pun aplicația dezvoltată în anumite condiții limită. Aceste metrici au în vedere analiza robustețiii aplicației și a algoritmilor dezvoltați în cadrul acesteia.

Unul dintre cele mai simple moduri de evidențiere a unor astfel de metrici este utilizarea de grafice, diagrame, tabele, etc. care permit persoanei care observă datele prezentate un mod rapid și eficient de analiză a acestora.

Instrumente pentru crearea de grafice

Pentru crearea de grafice există diverse instrumente care pot fi folosite pe lângă utilizarea arhicunoscutelor aplicații bazate pe spreadsheet-uri, precum Excel, pentru a crea diverse grafice simple. Unul dintre acestea este gnuplot [http://www.gnuplot.info/] care este un utilitar gratuit pentru linia de comandă ce poate fi rulat în cadrul a multiple sisteme de operare care a evoluat în timp pentru a oferi mai multe funcționalități. Unul dintre pachetele dezvoltate pe baza gnuplot este GNU Octave [https://octave.org/#] care este biblioteca folosită de către Matlab [https://www.mathworks.com/products/matlab.html] pentru realizarea de grafice. O altă bibliotecă utilă, gratuită, este matplotlib [https://matplotlib.org/] care este gândită mai mult pentru limbajul de dezvoltare python, dar care are diverse wrappere și pentru alte limbaje (ex: C++).

Indiferent de instrumentul utilizat în realizarea de grafice este important ca titlul, respectiv axele să fie etichetate în mod potrivit, cu nume sugestive, eventual pentru un graf cu mai multe variabile analizate să fie creată și o legendă care să specifice ce reprezintă fiecare variabilă.

mps/laboratoare/laborator-07.txt · Last modified: 2022/11/22 11:03 by mihai_lucian.voncila